Topic 7: Classifying Texts

LSE MY459: Computational Text Analysis and Large Language Models

https://lse-my459.github.io/

Ryan Hübert
Associate Professor of Methodology

Slides last updated: 1 March 2026

Supervised learning overview

Classifying into known categories

Our goal today: classify documents into known categories

We already (sort of) did this last week using ADMs

- ➜ Advantage of ADMs: not corpus-specific, so cost to apply to a new corpus is trivial
- ➜ Disadvantage of ADMs: not corpus-specific, so performance on a new corpus is unknown

Supervised learning is another approach:

- ➜ An alternative to ADMs where associations between features and categories are learned from data (and *not* from dictionary)
- ➜ Typically outperform ADMs for classification tasks when sufficient high-quality dataset is available

ADMs versus supervised learning

Barberá et al. (2021) codes a set of >4,000 articles, showing ML generally does better than ADMs coding sentiment

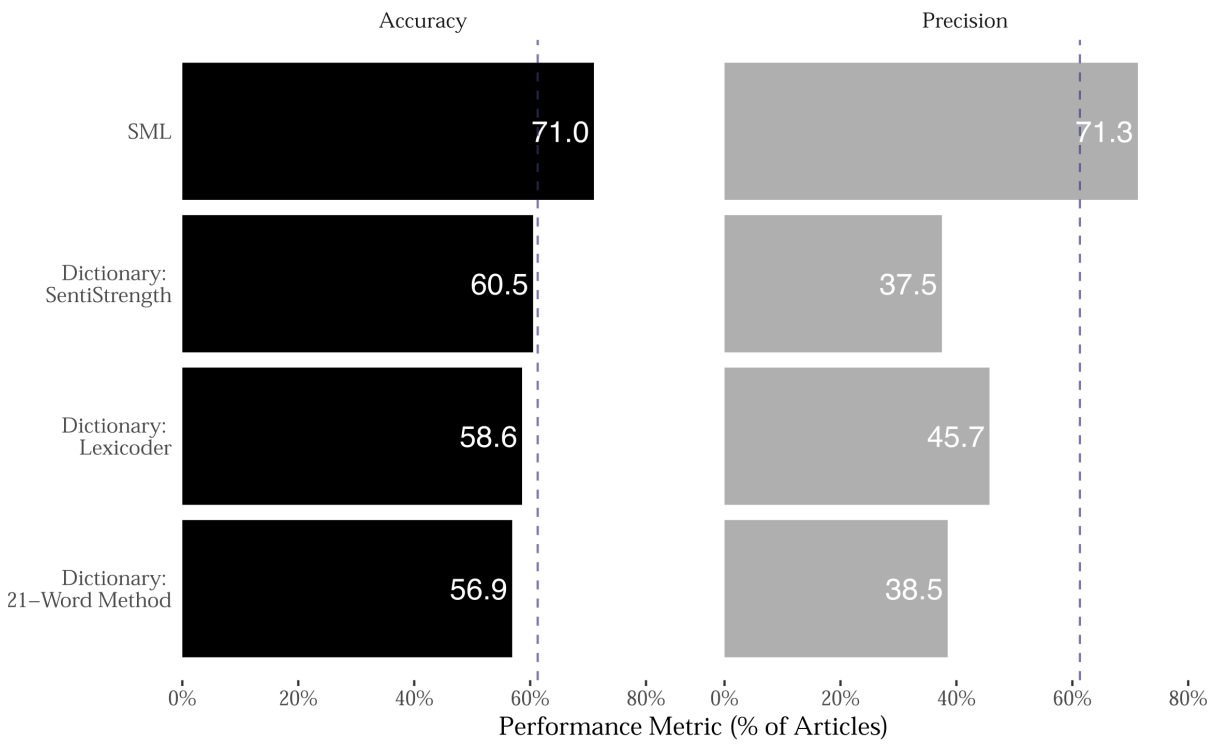


Figure 3. Performance of SML and Dictionary Classifiers—Accuracy and Precision.
Note: Accuracy (percent correctly classified) and precision (percent of positive predictions that are correct) for the ground truth dataset coded by ten CrowdFlower coders. The dashed vertical lines indicate the baseline level of accuracy or precision (on any category) if the modal category is always predicted. The corpus used in the analysis is based on the keyword search of *The New York Times* 1980–2011 (see the text for details).

Source: http://doi.org/10.1017/pan.2020.8

Backing up: why do this?

Classification with pre-defined categories is about *measurement*

A typical use case is **filling in missing data**, e.g.:

- ➜ Have large collection of documents that we want to partition into pre-defined categories
- ➜ Some documents are (or can be) **labelled**
- ➜ Many others are **unlabelled**

Supervised learning: use labelled data to build a model that we can use to "predict" labels for unlabelled data

- ➜ Idea: methods "supervised" by the existing labels

Can be useful for dealing with missing data (labels) in existing data and predicting categories in future data

Supervised v. unsupervised methods

This week: we're grouping documents ("classifying" them)

We also grouped documents in week 4: clustering, topic modelling

- ➜ Key difference: we didn't know groups ex ante

That was **unsupervised learning**, which is largely (but not always) about *discovering* categories of interest, not *measuring* pre-existing categories of interest

Key distinctions:

- ➜ Supervised methods: pre-defined categories, some documents are labelled
- ➜ Unsupervised methods: no pre-defined categories, no documents are labelled

Several issues we need to cover

Suppose we have a corpus containing documents we want to classify

1. How do we create (or find) a set of labelled documents?
   - ➜ What are the categories of interest and what is the best way to label a subset of documents?
2. How do we do the classification?
   - ➜ What classifier do we use?
   - ➜ How do we ensure the model generalises to new data?
3. How do we evaluate whether our classifier worked well?
   - ➜ What quantitative performance metrics can we use to evaluate?

Basic workflow

1. Gather documents
2. Create a training set by, e.g., hand labelling a random sample
3. Preprocess and quantify documents (e.g., to a dfm)
4. Choose a classifier and specify parameters
   - ➜ A "model" that you will estimate
5. Train the model
   - ➜ Output is an *estimated* model
6. Evaluate model performance
7. Use estimated model to generate *predicted* classifications for unlabelled documents

Creating a labelled set and evaluating its reliability

How do we obtain a labelled set?

1. **External sources of labels**

   - ➜ Documents authored by same people, like Federalist papers
   - ➜ Party labels for election manifestos
   - ➜ Legislative proposals by think tanks

2. **Expert labels**

   - ➜ "Canonical" dataset in Comparative Manifesto Project
   - ➜ In most projects, undergraduate students whose expertise comes from careful training

3. **Crowd-sourced labels**

   - ➜ **Wisdom of crowds**: aggregated judgements of non-experts converge to judgements of experts at much lower cost
   - ➜ Easy to implement with online platforms

Evaluating the quality of a labelled set

Before using labelled documents for classification, you *must* assess the quality of the labelled set

Considerations for a hand-coding from Neuendorf (2017):

- *Objectivity-intersubjectivity*: shared understanding among coders
- *A priori design*: "codebook" developed in advance of coding
- *Reliability*: different coders yield same codings
- *Validity*: coding captures what we intend it to
- *Genralisability*: can be used in new contexts
- *Replicability*: coding should be able to be replicated

One I'll add: is your labelled set *representative*?

See:
Krippendorff, Klaus. 2019. *Content Analysis: An Introduction to Its Methodology* 4th ed. SAGE Publications, Inc. https://doi.org/10.4135/9781071878781
Neuendorf, Kimberly A. 2017. *The Content Analysis Guidebook* 2nd ed. SAGE Publications, Inc. https://doi.org/10.4135/9781071802878

Evaluating the quality of a labelled set

GRS offers a good procedure for hand labelling

A key point: have multiple people label each document and then check the **inter-rater reliability** (or **inter-coder reliability**)

- ➜ **Percent agreement** is very simple and straight forward
- ➜ **Correlation**, usually Pearson's r, but can use others
- ➜ **Agreement measures** which factor in possibility of agreement by chance, such as Cohen's kappa or Krippendorff's alpha

For all: 1 is perfect agreement

Classifier performance metrics

Quantifying the performance of classifiers

Classifiers predict which class each document is in

Classifiers are built using labelled data where we already know the classifications

- We think of the already existing labels as the "truth"

To evaluate performance we ask: how good of a job did the classifier do predicting classes, when compared to the truth?

To build ideas, consider a simple example:

- A classification task with three classes, A, B and C
- 120 "labelled" documents (i.e., we know the class)
- A classifier that predicted each document's class

Confusion matrix

A **confusion matrix** contains counts of documents that are (1) truly in each class and (2) predicted to be in each class by the classifier

		Predicted class			
		A	B	C	
True class	A	**35**	10	5	**50**
	B	6	**24**	9	**39**
	C	8	7	**16**	**31**
		49	**41**	**30**	**120**

With this matrix, you can calculate several metrics

Standard performance metrics

Accuracy: how correct is the classifier's identifications overall?

$$\text{Accuracy} = \frac{\text{Correctly classified}}{\text{Total number classified}}$$

How correct is my classifier's classifications of a particular class?

- **Precision** (for a class k): what proportion of observations classified in class k are truly in that class?

$$\text{Precision}_k = \frac{\text{Correctly classified as } k}{\text{Total number classified as } k}$$

- **Recall** (for a class): what proportion of observations in the class are correctly classified by the classifier?

$$\text{Recall} = \frac{\text{Correctly classified as } k}{\text{Total number truly in class } k}$$

Standard performance metrics

For a class k, you can also construct a composite measure of precision and recall (technically, the harmonic mean)

$$\text{F1}_k = 2 \times \frac{\text{Precision}_k \times \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}$$

And since precision and recall are defined on a class-by-class basis, you can also create an F1 that is an average across classes

- ➜ You have to decide how to average for your specific purpose
  See https://en.wikipedia.org/wiki/F-score#Extension_to_multi-class_classification

Warning: lots of people report precision, recall and F1 without explicitly saying which class – use context clues to figure it out

Confusion matrix

		Predicted class			
		A	B	C	
True class	A	**35**	10	5	**50**
	B	6	**24**	9	**39**
	C	8	7	**16**	**31**
		49	**41**	**30**	**120**

What is the accuracy of this classifier?

$$\text{Accuracy} = \frac{35 + 24 + 16}{120} = 0.625$$

In other words: what proportion of the documents were classified correctly by the classifier?

Confusion matrix

True class \ Predicted class	A	B	C	
A	**35**	10	5	**50**
B	6	**24**	9	**39**
C	8	7	**16**	**31**
	49	**41**	**30**	**120**

What is the precision of this classifier for class A?

$$\text{Precision}_A = \frac{35}{35+6+8} = 0.714$$

In other words: of the documents that my classifier classified as A, what proportion were actually in class A?

Confusion matrix

		Predicted class			
		A	B	C	
True class	A	**35**	10	5	**50**
	B	6	**24**	9	**39**
	C	8	7	**16**	**31**
		49	**41**	**30**	**120**

What is the recall of this classifier for class A?

$$\text{Recall}_A = \frac{35}{35 + 10 + 5} = 0.700$$

In other words: of the documents that are in class A, what proportion were classified correctly by the classifier?

Confusion matrix

True class \ Predicted class	A	B	C	
A	**35**	10	5	**50**
B	6	**24**	9	**39**
C	8	7	**16**	**31**
	49	**41**	**30**	**120**

Can calculate precision and recall for the other classes too:

$$\text{Precision}_B = \frac{24}{10+24+7} = 0.585 \qquad \text{Recall}_B = \frac{24}{6+24+9} = 0.615$$

$$\text{Precision}_C = \frac{16}{5+9+16} = 0.533 \qquad \text{Recall}_C = \frac{16}{8+7+16} = 0.516$$

Which performance metric do I use?

Deciding which metric to focus on depends on your task

Example from GRS: spam filter

- ➜ If you want to make sure that messages sent to junk folder are actually spam: *precision*
- ➜ If you want to make sure that all spam ends up in the junk folder: *recall*

For an email provider, precision is (probably) more important

Report multiple metrics as they give different information

- ➜ Accuracy is often misleading...

Why you should report multiple statistics: spam

		Predicted class		
		Not Spam	Spam	
True class	Not Spam	**920**	60	**980**
	Spam	20	**20**	**40**
		940	**80**	**1020**

$$\text{Accuracy} = \frac{940}{1020} = 0.92$$

$$\text{Recall}_S = \frac{20}{20+20} = 0.50$$

$$\text{Precision}_S = \frac{20}{20+60} = 0.25$$

92% are accurately classified, but 50% of spam messages are getting through the filter and 75% of messages in the junk mail folder *aren't spam*(!)

Why you should report multiple statistics: illegal content

		Predicted class: Legal	Predicted class: Illegal	
True class	Legal	**850**	50	**900**
	Illegal	100	**50**	**150**
		950	**100**	**1050**

$$\text{Accuracy} = \frac{900}{1050} = 0.86$$

$$\text{Recall}_I = \frac{50}{50 + 100} = 0.33$$

$$\text{Precision}_I = \frac{50}{50 + 50} = 0.50$$

86% are accurately classified, but 66% of illegal content is getting through the filter and 50% of blocked content isn't illegal

Binary classification

Binary classification tasks are common, e.g.

- Spam versus not spam
- Illegal content versus legal content
- Republican speech versus Democratic speech

We have some special (but confusing) terminology:

		Predicted class	
		Positive	Negative
True class	Positive	True Positives	False Negatives (Type II error)
	Negative	False Positives (Type I error)	True Negatives

The confusing part: what's "positive" and what's "negative"?

Examples of classifiers: Naive Bayes

Doing the classification

So far, we've talked a lot about classifier performance

In some sense, we were getting ahead of ourselves

Now we ask: *how do we actually create a classifier?*

There are a set of algorithms that are designed for this

- ➜ When applied to a dataset, they generate estimated "scores" for each document and each class
- ➜ Often, these scores can be interpreted as probabilities, e.g., probability that document i is in class k
- ➜ Once you get these estimated scores, you need to decide how to classify each document into a class

The example set up

Suppose we have N documents, each of which we want to classify into *one* of K classes

- For every document i, $\pi_{ik} = 1$ if document i is in class k and $\pi_{ik} = 0$ otherwise
- Some documents are labelled (π_{ik} is known for all k), while other documents are unlablled

Labelled documents are used to **train** a classifier

- Classifier learns patterns in the labelled documents
- Gives us an estimated model, allowing us to approximate $\widehat{\Pr}(\pi_{ik} = 1 | D_i)$ for each document D_i
- Can use model to predict $\widehat{\pi}_{ik}$ for all k (classify)

What is our goal?

We can't observe $\Pr(\pi_{ik} = 1|D_i)$ directly, but using Bayes' theorem, we can write:

$$\Pr(\pi_{ik} = 1|D_i) = \frac{\Pr(\pi_{ik} = 1)\Pr(D_i|\pi_{ik} = 1)}{\Pr(D_i)}$$

Since our goal is classification, we can drop so-called "scaling" factors that do not depend on k—here, the denominator

$$\Pr(\pi_{ik} = 1|D_i) \propto \underbrace{\Pr(\pi_{ik} = 1)}_{\text{class prevalence (prior)}} \overbrace{\Pr(D_i|\pi_{ik} = 1)}^{\text{likelihood under class-specific language model}}$$

Note: $\propto$ means "proportional to"

Specifying a model

The quantity $\Pr(D_i|\pi_{ik} = 1)$ is a class-specific language model evaluated at document D_i

To estimate, we need to say which *specific* language model

Lots of directions we could go, but with a DFM and bag of words, we can use a multinomial model (like before!)

- Document D_i is represented by $\mathbf{W}_i$ (vector of feature counts)
- Document D_i has $M_i = \sum_j W_{ij}$ tokens (sum across columns)

Since the model depends on the class π_{ik}, we can write

$$\mathbf{W}_i|(\pi_{ik} = 1) \sim \text{Multinomial}(M_i, \boldsymbol{\mu}_k)$$

Specifying a model

The multinomial likelihood is (from Wikipedia):

$$\Pr(\mathbf{W}_i|\pi_{ik}=1) = \frac{M_i!}{\prod_{j=1}^{J}(W_{ij}!)}\prod_{j=1}^{J}\left(\mu_{kj}^{W_{ij}}\right)$$

We can, again, drop the multinomial coefficient because it is a scaling factor:

$$\Pr(\mathbf{W}_i|\pi_{ik}=1) \propto \prod_{j=1}^{J}\left(\mu_{kj}^{W_{ij}}\right)$$

Specifying a model

Now recall what we're interested in:

$$\Pr(\pi_{ik} = 1|D_i) \propto \Pr(\pi_{ik} = 1)\Pr(D_i|\pi_{ik} = 1)$$

Given we assumed the multinomial model, we can substitute:

$$\Pr(\pi_{ik} = 1|D_i) \propto \Pr(\pi_{ik} = 1)\prod_{j=1}^{J}\left(\mu_{kj}^{W_{ij}}\right)$$

And finally, let's use α_k to indicate the prevalence of class k:

$$\Pr(\pi_{ik} = 1|D_i) \propto \alpha_k\prod_{j=1}^{J}\left(\mu_{kj}^{W_{ij}}\right)$$

Naive Bayes classifier

This is the full model we plan to estimate and use as our classifier

$$\Pr(\pi_{ik} = 1 | D_i) \propto \alpha_k \prod_{j=1}^{J} \left(\mu_{kj}^{W_{ij}} \right)$$

With multinomial model: we assume that conditional on class k, tokens are drawn independently from $\boldsymbol{\mu}_k$

- This is the "naive" assumption in Naive Bayes
- Other distributional assumptions may allow for token draws to be correlated—no longer Naive Bayes + harder to estimate
- The model doesn't estimate $\Pr(\pi_{ik} = 1 | D_i)$, but rather a "score" that is proportional to this probability

Estimating a Naive Bayes classifier

Remember what we're interested in estimating:

$$\Pr(\pi_{ik} = 1 | D_i) \propto \alpha_k \prod_{j=1}^{J} \left(\mu_{kj}^{W_{ij}} \right)$$

There are two quantities that we can estimate: $\boldsymbol{\alpha}$ and $\boldsymbol{\mu}$

- $\boldsymbol{\alpha}$ is a vector of all the class prevalences (the α_k parameters)
- $\boldsymbol{\mu}$ is a matrix containing all the individual μ_{kj} parameters

We can calculate all these things with our labelled data

How?

Estimating a Naive Bayes classifier

What proportion of labelled documents are in class k?

$$\hat{\alpha}_k = \frac{\sum_i \pi_{ik}}{N}$$

What proportion of all tokens used in labelled documents in class k are token j?

$$\hat{\mu}_{kj} = \frac{\sum_i \pi_{ik} W_{ij}}{\sum_j \sum_i \pi_{ik} W_{ij}}$$

There's a problem here: the numerator can be zero if feature j is never observed in class k (in the training data)

- ➜ Means that we'll never predict class k
- ➜ Need to do smoothing

Estimating a Naive Bayes classifier

Simplest solution is to add a fixed smoother of c:

$$\hat{\mu}_{kj} = \frac{c + \sum_i \pi_{ik} W_{ij}}{\sum_j \left(c + \sum_i \pi_{ik} W_{ij}\right)} = \frac{c + \sum_i \pi_{ik} W_{ij}}{Jc + \sum_j \sum_i \pi_{ik} W_{ij}}$$

(You can also add a Dirichlet prior—see GRS)

In this case, the value of c is a **hyperparameter** of the model: a "setting" that we choose *before* estimating the model

- ➜ Hyperparameters control properties of the model
- ➜ Quality of predictions depends on choosing them well

Estimating a Naive Bayes classifier

For a document i, we can estimate the model with

$$\widehat{\Pr}(\pi_{ik} = 1 | \mathbf{W}_i) \propto \hat{\alpha}_k \prod_j \hat{\mu}_{kj}^{W_{ij}}$$

This will yield a quantity that is proportional to the probability that a document $\mathbf{W}_i$ is in class k

- ➜ Recall we dropped two scaling factors
- ➜ Again, this is a "score"

To make it an actual probability:

$$\widehat{\Pr}(\pi_{ik} = 1 \mid \mathbf{W}_i) = \frac{\hat{\alpha}_k \prod_j \hat{\mu}_{kj}^{W_{ij}}}{\sum_{\ell=1}^{K} \hat{\alpha}_\ell \prod_j \hat{\mu}_{\ell j}^{W_{ij}}}.$$

Classifying with an estimated Naive Bayes classifier

Once you have estimated the "score" $\hat{\alpha}_k \prod_j \hat{\mu}_{kj}^{W_{ij}}$ for each document i and class k, how do you classify?

We can esimate each documents class:

$$\hat{\pi}_{ik} = \begin{cases} 1 & \text{if } k = \arg\max_\ell \hat{\alpha}_\ell \prod_j \hat{\mu}_{\ell j}^{W_{ij}} \\ 0 & \text{otherwise} \end{cases}$$

In words: $\hat{\pi}_{ik} = 1$ if the estimated score for k is the highest of all possible classes

An example: inauguration speeches

An example of a (reduced) DFM from the inaugural address corpus

Suppose we want to classify whether a speech was written by Trump using data from all speeches from 2000 to 2021

	pi.ik	america	us	nation
2001-Bush	0	11	11	14
2005-Bush	0	20	3	13
2009-Obama	0	10	23	15
2013-Obama	0	9	21	9
2017-Trump	1	19	2	13
2021-Biden	0	20	27	16

Right away, we can calculate

$$\hat{\alpha}_{\mathcal{T}} = \frac{1}{6} \approx 0.167 \quad \text{and} \quad \hat{\alpha}_{\mathcal{O}} = \frac{5}{6} \approx 0.833$$

An example: inauguration speeches

Collapsing the DFM into the two categories:

	pi.ik	america	us	nation
Not.Trump	0	70	85	67
Trump	1	19	2	13

Let's calculate the probability Trump uses word `america` ($\hat{\mu}_{\mathcal{T}_{\texttt{a}}}$):

$$\hat{\mu}_{\mathcal{T}_{\texttt{a}}} = \frac{c + 19}{(c + 19) + (c + 2) + (c + 13)}$$

Adding $c = 1$ as a smoother yields:

$$\hat{\mu}_{\mathcal{T}_{\texttt{a}}} = \frac{20}{37} \approx 0.541$$

(This is an arbitrary c, better to "tune"—more later)

An example: inauguration speeches

Collapsing the DFM into the two classes:

	pi.ik	america	us	nation
Not.Trump	0	70	85	67
Trump	1	19	2	13

Let's calculate the probability other presidents use word `america` ($\hat{\mu}_{\mathcal{O}\mathsf{a}}$):

$$\hat{\mu}_{\mathcal{O}\mathsf{a}} = \frac{c + 70}{(c + 70) + (c + 85) + (c + 67)}$$

Adding $c = 1$ as a smoother yields:

$$\hat{\mu}_{\mathcal{O}\mathsf{a}} = \frac{71}{225} \approx 0.316$$

An example: inauguration speeches

Repeat for each feature to get all estimated model parameters

	$\hat{\alpha}_k$	$\hat{\mu}_{k\mathsf{a}}$	$\hat{\mu}_{k\mathsf{u}}$	$\hat{\mu}_{k\mathsf{n}}$
Trump	0.167	0.541	0.081	0.378
Not Trump	0.833	0.316	0.382	0.302

Now, let's predict the category for the first Bush speech using the model parameters

```
              pi.ik     america          us      nation
2001-Bush         0          11          11          14
```

$$\widehat{\Pr}(\text{Trump}|\mathbf{W}_{\text{B2001}}) \propto 0.167 \times 0.541^{11} \times 0.081^{11} \times 0.378^{14} = 2.355282 \times 10^{-22}$$

$$\widehat{\Pr}(\text{Not Trump}|\mathbf{W}_{\text{B2001}}) \propto 0.833 \times 0.316^{11} \times 0.382^{11} \times 0.302^{14} = 3.472943 \times 10^{-18}$$

Classification: Not Trump since the NB score is higher

Elephant in the room

Weird thing about this example: we don't have any unlabelled data

Here: just focused on process of creating a classifer by estimating it from labelled data

- This is pedagogically useful

Ultimate goal would be prediction for unlabeled data

For example:

- Suppose you have a large corpus of presidential speeches
- The corpus does not track which speech was given by which president
- Classifier can help you estimate this

(Even so: a bit artificial)

Another example

▶ **Table 13.1** Data for parameter estimation examples.

	docID	words in document	in $c = China$?
training set	1	Chinese Beijing Chinese	yes
	2	Chinese Chinese Shanghai	yes
	3	Chinese Macao	yes
	4	Tokyo Japan Chinese	no
test set	5	Chinese Chinese Chinese Tokyo Japan	?

Source: Manning, Christopher D., Raghavan, Prabhakar, and Schütze, Hinrich. 2008. *Introduction to information retrieval*. Cambridge University Press. LSE Library link

Another example

Example 13.1: For the example in Table 13.1, the multinomial parameters we need to classify the test document are the priors $\hat{P}(c) = 3/4$ and $\hat{P}(\overline{c}) = 1/4$ and the following conditional probabilities:

$$\begin{aligned}
\hat{P}(\text{Chinese}|c) &= (5+1)/(8+6) = 6/14 = 3/7 \\
\hat{P}(\text{Tokyo}|c) = \hat{P}(\text{Japan}|c) &= (0+1)/(8+6) = 1/14 \\
\hat{P}(\text{Chinese}|\overline{c}) &= (1+1)/(3+6) = 2/9 \\
\hat{P}(\text{Tokyo}|\overline{c}) = \hat{P}(\text{Japan}|\overline{c}) &= (1+1)/(3+6) = 2/9
\end{aligned}$$

The denominators are $(8+6)$ and $(3+6)$ because the lengths of $text_c$ and $text_{\overline{c}}$ are 8 and 3, respectively, and because the constant B in Equation (13.7) is 6 as the vocabulary consists of six terms.

We then get:

$$\begin{aligned}
\hat{P}(c|d_5) &\propto 3/4 \cdot (3/7)^3 \cdot 1/14 \cdot 1/14 \approx 0.0003. \\
\hat{P}(\overline{c}|d_5) &\propto 1/4 \cdot (2/9)^3 \cdot 2/9 \cdot 2/9 \approx 0.0001.
\end{aligned}$$

Thus, the classifier assigns the test document to $c = China$. The reason for this classification decision is that the three occurrences of the positive indicator Chinese in d_5 outweigh the occurrences of the two negative indicators Japan and Tokyo.

Examples of classifiers: Regularised regression

Regularised regression

Another common classification algorithm is **linear regression**: choose $\boldsymbol{\mu}_1, ..., \boldsymbol{\mu}_K$ to minimise residual sum of squared errors:

$$\text{RSS} = \sum_{i=1}^{N} \sum_{k=1}^{K} (\pi_{ik} - \mathbf{W}_i \boldsymbol{\mu}_k)^2$$

If $\boldsymbol{\mu}_k = (\mu_{0k}, \mu_{1k}, ..., \mu_{Jk})$, we can write this as:

$$\sum_{i=1}^{N} \sum_{k=1}^{K} \left(\pi_{ik} - \mu_{0k} - \sum_{j=1}^{J} \mu_{jk} W_{ij} \right)^2$$

We'll focus on binary classification so that $\pi_i \in \{0, 1\}$, and $\boldsymbol{\mu} = (\mu_0, \mu_1, ..., \mu_J)$

$$\text{RSS} = \sum_{i=1}^{N} \left(\pi_i - \mu_0 - \sum_{j=1}^{J} \mu_j W_{ij} \right)^2$$

Regularised regression

Standard linear regression doesn't work well for DFMs

- If $J > N$, then OLS does not have a unique solution
- Even when $N > J$, linear regression models can be "too sensitive" to patterns in the labelled documents

To deal with these issues, we will use **regularised (linear) regression**:

- Regularisation: adding a penalty for model complexity
- In our case, model complexity = too many features
- Penalty will down-weight highly predictive features

Regularised regression

Ridge regression applies an **L2 penalty**:

$$\sum_{i=1}^{N}\left(\pi_i - \mu_0 - \sum_{j=1}^{J}\mu_j W_{ij}\right)^2 + \lambda\sum_{j=1}^{J}\mu_j^2$$

Lasso regression applies an **L1 penalty**:

$$\sum_{i=1}^{N}\left(\pi_i - \mu_0 - \sum_{j=1}^{J}\mu_j W_{ij}\right)^2 + \lambda\sum_{j=1}^{J}|\mu_j|$$

λ is the **penalty parameter**

- ➜ It is a hyperparameter (like the smoother c in Naive Bayes) that must be chosen before estimation

Elastic net regression: weighted average of L1 and L2 penalties

Lasso vs. ridge regression

How does the penalty work?

- Algorithm is trying to *minimise* RSS but λ is increasing it
- The optimal way to minimise RSS, given the penalty: push down coefficients that create large penalties
- If $\lambda = 0$, both are equivalent to standard OLS

Both types of regression penalise large regression coefficients

- The idea: we're worried about some features causing our model predictions to swing wildly (too much variance)
- Features with huge effects are pushed toward zero

Lasso pushes some coefficients to zero, but ridge does not

- Lasso can be used for feature selection and to identify a regression when $J > N$

Regularised regression

Estimation gives us $\hat{\boldsymbol{\mu}} = (\hat{\mu}_0, \hat{\mu}_1, ..., \hat{\mu}_J)$

We don't care about these estimates per se – our goal is to classify into classes

For each document i, we can calculate a "score":

$$\hat{\mu}_0 + \sum_{j=1}^{J} \hat{\mu}_j W_{ij}$$

This score *approximates* the probability that document i is classified as 1

➜ Can use logit if you want a probability

Classification with scores

How do you determine document i's class using these scores?

One intuitive option:

- ➜ Classify document i in a class k if the predicted score is greater than 0.5

But using 0.5 doesn't usually work well

- ➜ Remember: scores only approximate probabilities
- ➜ But also, data can be quite unbalanced

For example: if you are trying to classify unbalanced classes (e.g., illegal content), 0.5 threshold is not going to work

- ➜ All your scores will be very low or very high

Advantages and disadvantages of regularised regression

Advantages of regularised regression:

- ➜ Easy to interpret
- ➜ Works well when features are each independently informative
- ➜ Works well when data is limited

Disadvantages:

- ➜ Does not work well when features are, e.g., jointly informative
- ➜ Often outperformed by more complex methods

Adaptive basis function methods: classifiers that adaptively learn feature importance (tree methods, boosting, neural networks)

- ➜ These topics are covered in more detail in MY474

Building a “good” classifier

Choosing your classifier

Which classifier should you use?

Obviously, you want to use a classifier that performs the "best" (e.g., high accuracy, recall and precision)

You have to choose which classifier to use:

- ➜ Which algorithm? E.g. regularised regression or naive Bayes
- ➜ What hyperparameters do you want to set?

This process is called **model selection**

It's tempting to try a lot of options and pick the best performer using performance metrics—but beware...

Overfitting

Classifiers are optimised to maximise performance statistics within the **training sample** used to do the classification

- ➜ In otherwords: they are optimised for **in-sample performance**

But generally we want to apply our models to new data

- ➜ For example, we want to classify a large number of documents into classes, but only have a small labelled sample

Overfitting occurs when a classifier builds a model that is too closely tailored for the sample used for training

- ➜ Overfitting means a classifier won't perform well on new data

Overfitting

Contrived example:

	by	man	upon
Hamilton	859	102	374
Jay	82	0	1
Madison	474	17	7

Suppose a classifier generates this estimated model:

- A document is written by Hamilton if and only if it contains 859 uses of the word "by"; written by Jay if and only if it contains 82 uses of the word "by"; and written by Madison if and only if it contains 474 uses of the word "by"

This model will perform *perfectly* in the training set: but is it a good model for use in other data? It has **overfit** to the sample!

- Roughly speaking, it is taking the sampling noise too seriously

Overfitting

Can happen when the model you want to estimate is "too complex"

- Machines are very good at finding patterns, even spurious ones
- Giving your machine a lot of flexibility (by letting it fit a very complex model) allows it more room to find patterns
- But it's finding patterns specific to the training set, so its performance in-sample will (mechanically) be better than its performance **out-of-sample**
- You need to discipline the machine because you want to use the model for out-of-sample classification!

Overfitting can also happen with sparse datasets (like DFMs)

Test sets

How can we see whether an estimated model performs well on other datasets than the training set?

We *test* it on other datasets!

A **test set** is a subset of our labelled data that we "hold out"

- ➜ The machine is not allowed to look at it, or use it in any way

The machine builds its model using training data only

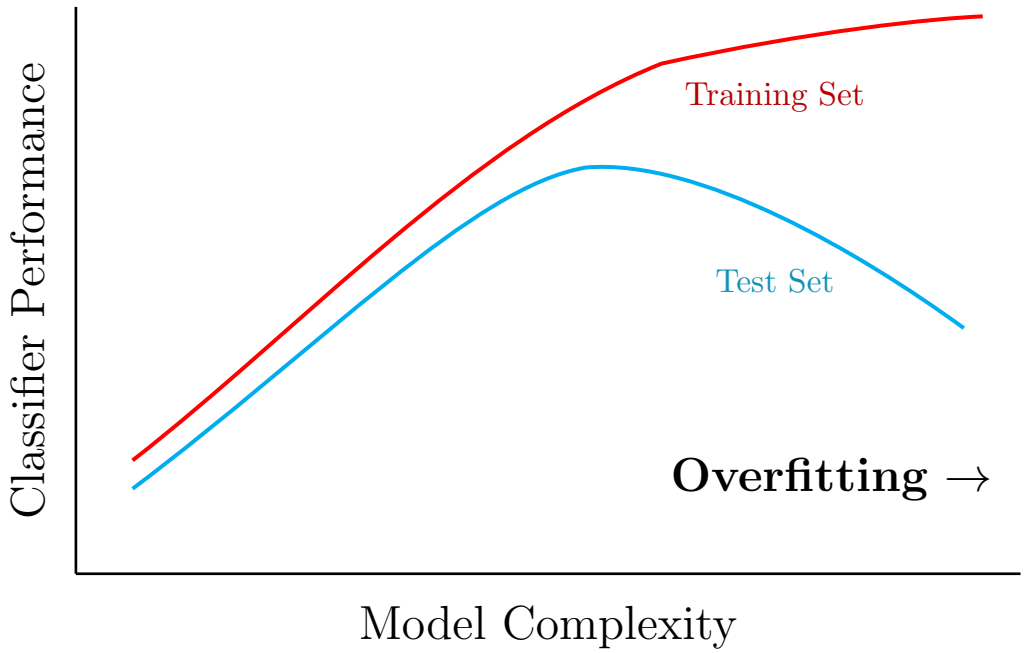
- ➜ It probably will overfit to the training data (at least a little)
- ➜ But we can see how much by looking at how well model does on test data

Back to choosing your model

In-sample performance statistics will be misleading because classifiers will overfit to the training set

But can they still be used to choose the best classifier?

It's complicated:



Back to choosing your model

Instead of choosing a classifier based on in-sample performance, suppose you base it on test set performance:

- Create a test set for the purposes of evaluating performance of several approaches
- Train 10 different algorithms on the training set
- Evaluate performance of each on the test set
- Choose the algorithm that has highest performance in test set

This will *also* likely lead to overfitting

But why??

Choosing your model

You already know: classifier tends to overfit to training data

- ➜ It does so by making choices based on how well its model performs in the training data
- ➜ This is unavoidable

But by evaluating several models on the test set, the human is now doing the same thing:

- ➜ She is making model selection decisions based on performance in the test set
- ➜ And thus, tailoring the model to the test set

Oops!

Choosing your model

This seems impossible

- ➜ On the one hand, we want the best performing model
- ➜ On the other hand, we can't figure out which one is the best performing without potentially causing overfitting

At some level, this is just a hard problem

But there are some things we can do:

1. Validation
2. Choose approaches we *know* will do better without testing
   - ➜ Think hard about model selection
   - ➜ **Tune** your hyperparameters

Validation

In addition to training and test sets, also create a validation set

- ➜ The validation set is used to evaluate your model performance and choose a model

A major advantage: it allows you to keep your test set untouched

Seems like a small thing, but it is key:

The test set is only used one time to evaluate a model at the very end once the model has been chosen and estimated.

You cannot try to train another model after looking at test set performance

At least now you will be able to see if you overfit!

Validation

Doesn't solve all our problems, potentially creates new one:

1. Same principles apply: can still overfit to the *validation set*
2. What if you (randomly) get an unusual validation set?
   - ➜ This will make overfitting even worse
3. You're eating into your training set, reserving more of it for validation
   - ➜ Machines aren't as good training with less data
   - ➜ Limits the kinds of models you can train
   - ➜ Specifically: can't fit useful complex models

Validation

A core issue with this validation approach: wasting data

- ➜ You didn't use all your data to train, nor did you validate your model on all your data

If you validate on all* your data:

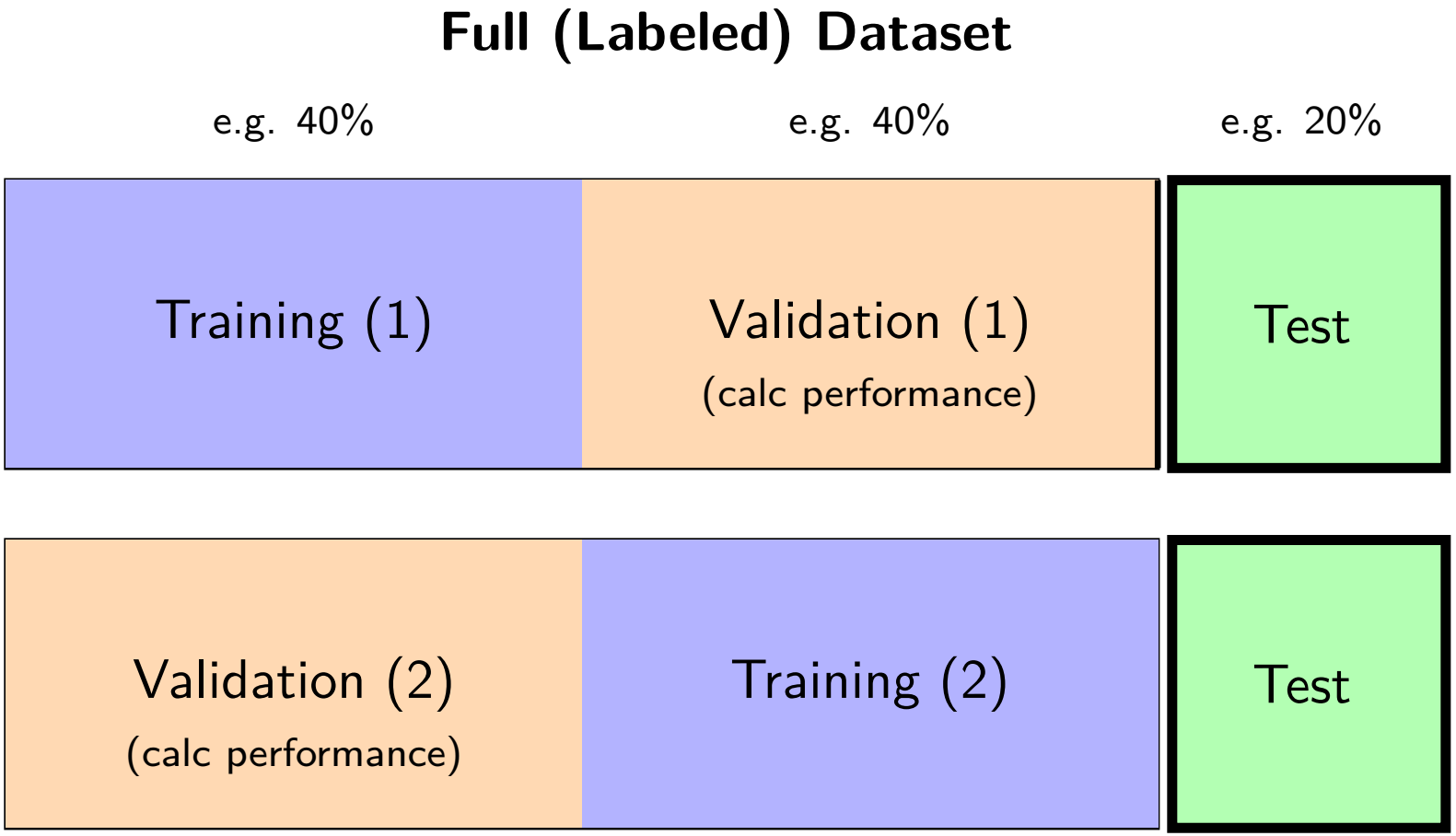
- ➜ You're guarding against drawing a "weird" validation set

If you train with all* your data:

- ➜ You have a wider array of approaches and models you can try

{*} Note: always need to keep a test set held out and stored away

Cross-validation



Cross-validation

Model selection with **cross-validation**:

1. Split into training, validation and test
2. Pick an algorithm/model
3. Train on the training set and validate on the validation set then *switch* to train on the validation set and validate on the training set
4. Average performance metrics on validation sets
5. Repeat steps 2-4 for each algorithm/model you are considering
6. Choose the algorithm that has best average performance on validation sets
7. Quantify performance on the test set

Cross-validation

Now you've trained *and* validated the model on all the data

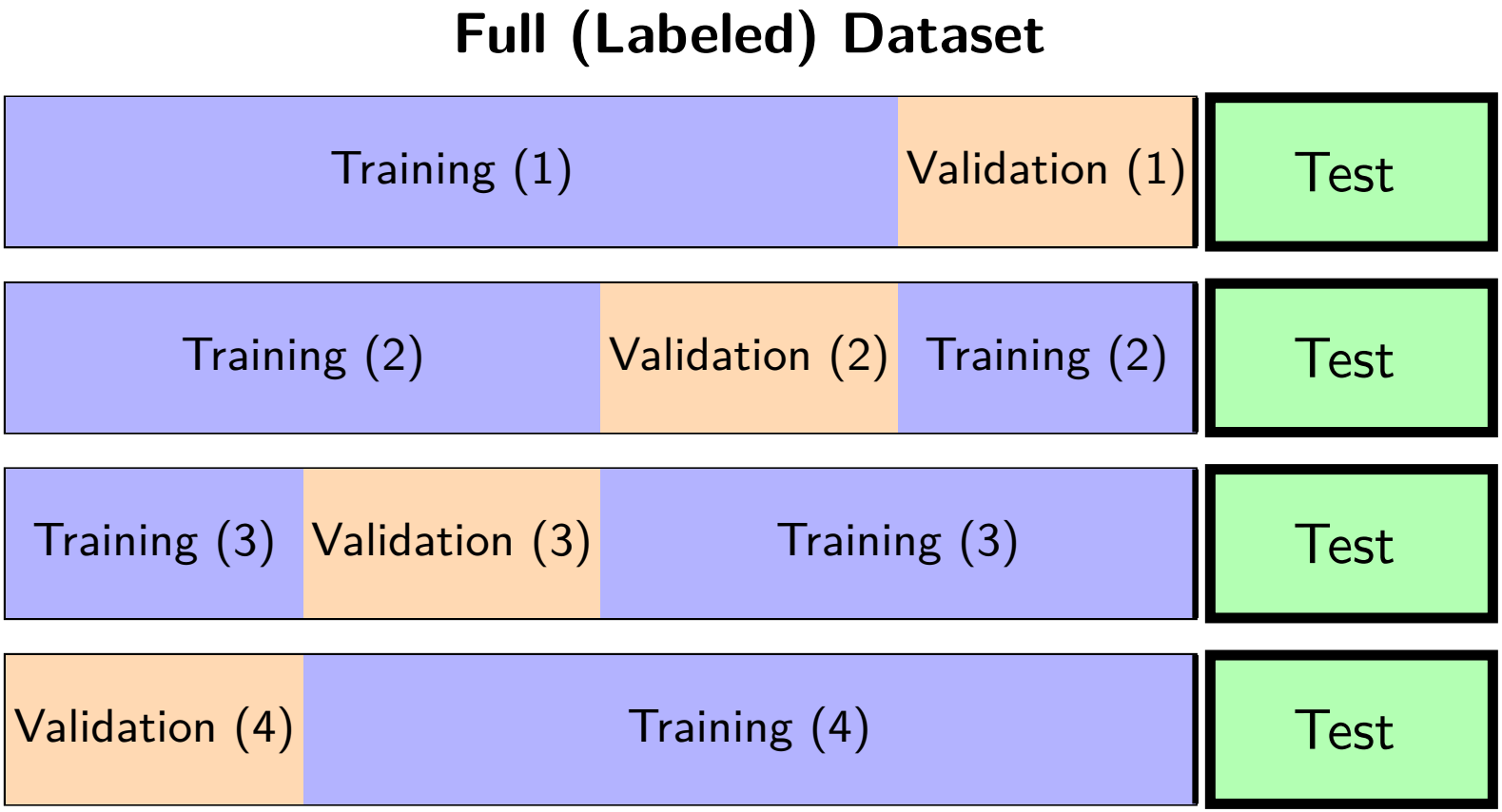
When you average out the performance across the two validation sets:

- ➜ "Unusually" high performance in one "unusual" set will be pulled down
- ➜ "Unusually" low performance in one "unusual" set will be pulled up

This helps you guard against overfitting during model selection

We can do *even better*: why only split the data once?

Cross-validation



Cross-validation

Notice: each round of training uses a bigger dataset

- This helps the machine out
- It is easier for it to learn with more data

I just showed you **four fold cross-validation** where you partition your data into 4 randomly created **folds**

- For each fold k, you treat all the folds *except* k as a training set, fitting a model on it and evaluating performance on the held-out fold k
- Repeat for each of the remaining folds
- Average together the performance metrics across the folds

More folds: (1) computation costs increase, but (2) each fold's training set is larger and more representative of full dataset

Cross-validation

Some bad news...

You can still create overfitting if you use cross-validation, then tune your model and use cross-validation again (esp. with same folds)

The principle remains unchanged: you are at risk of overfitting when you tune your model in response to measured performance

- E.g., iteratively training on same training set in response to performance statistics from previous rounds of training

But: cross-validation is an aggressive attempt to reduce overfitting without causing underfitting:

- Use larger training sets in each round (reduce underfitting)
- Average performance across each fold's validation set (guard against overfitting)

Choosing good models

Second thing you can do: make smart choices, such as:

- Choosing algorithms that work well for you kind of data (this involves a lot of research and thinking)
- Making smart pre-processing choices, e.g., weight and normalise for regression-based classifier
- **Tuning** hyperparameters to optimise performance:
  1. Define a grid of "candidate" values, such as $\lambda \in \{0.01, 0.1, 0.2, 0.5, 1, 5, 10\}$
  2. For each candidate, perform k-fold cross validation
  3. Set the hyperparameter value to the candidate value with best average validation performance
  4. Re-train the final model using the full training set and the selected hyperparameter value